

説明資料

「異世界観測者ノアの日記」

目次

1. 日記の読み方ガイド
2. 背景と目的
3. 課題意識
4. ランダム観測地点の着想
5. キャラクター設計
6. 生成フロー
7. 使用モデル・ツール・ライブラリ
8. 工夫した点
9. うまくいった点（定量データを含む）
10. 難しかった点
11. プロンプト設計の詳細
12. リポジトリ構成と実行手順
13. 計算資源
14. 今後の改善余地

1. 日記の読み方ガイド

本資料には「説明書（本ファイル）」と「観測日記まとめ.pdf」の2つのファイルがある。

■ 読む順番

1. まずこの説明書の「2. 背景と目的」「3. 課題意識」「4. ランダム観測地点の着想」「5. キャラクター設計」を読み、世界観を把握する。
2. 「観測日記まとめ.pdf」を Day 1 から順に読む。
3. 技術的な仕組みに興味があれば、この説明書の残りのセクションを読む。

■ 日記の構成（1日あたり）

各日の記録は以下の3層で構成されている。

- **日記本文:** ノアがその日に見た人間の情景描写。物語として読む部分。
- **この日に発見した法則:** 観察から導き出された「人間の生態の法則」。日記のどの場面から法則が生まれたか対応づけて読むと面白い。
- **観測者の内面:** ノア自身の内面の変化。Phase 1（観測者）からPhase 2（違和感）への遷移が、ここに最も色濃く表れる。

2. 背景と目的

人間が思わず興味を持ち、継続して読みたくなるAIキャラクターを設計する上で、「深い共感」が鍵になると考えた。単なる出来事の羅列や一般的な感情の吐露ではなく、「普段は意識していないが、言われてみれば

ば確かにそう感じる」という、日常に対する解像度の高い観察や、言語化されていなかった感情の機微をAIに持たせることで、人間にとってより魅力的でリアリティのある日記が成立すると考え、本課題に取り組んだ。

3. 課題意識

上記の目的を達成し、かつ1週間という時間の流れの中でキャラクターの輪郭を立ち上げるため、以下の3点をシステム上の主要な課題として設定した。

- **「成長する物語」の構築:** 単発の記憶を引き継ぐだけでなく、一連の経験を通じてキャラクターの認識や感情がどのように変化・成長していくかを、システムとしてどう表現するか。
- **長期的生成における単調さの回避:** LLMの生成を繰り返すうちに内容が均質化しやすいという問題に対し、いかにしてキャラクター特有の鋭い視点や「語り口の面白さ」を長期間維持し続けるか。
- **感情変化と記憶管理のスケーラビリティ:** 経過日数に応じたプロンプトの直接的な指定（例：「○日目はこう書く」といったハードコーディング）に頼らず、AIが自律的に状況を解釈し、記憶を参照しながら感情を変化させるスケーラブルなエージェント設計をどのように実現するか。

4. ランダム観測地点の着想

上記の「単調さの回避」と「深い共感」の両方を実現するアプローチとして、毎回ランダムな場所を観測地点に設定する仕組みを考えた。

人混みの多い繁華街、静かな住宅街、店が並ぶ商店街、人がほとんどいない路地裏——場所が変われば、そこで目にする人間の営みも変わる。同じキャラクターでも、観測する環境が異なれば異なる気づきや法則が生まれるため、日記の内容に自然な多様性をもたらすことができると考えた。

同時に、読み手にとっての面白さも意図している。生成された日記の舞台がたまたま自分の知っている場所——たとえば子供の頃の通学路や馴染みのある街——と重なったとき、自分の記憶とノアの観察が交差する。「あの場所を、この観測者はこう見るのか」という驚きが、より深い共感につながることを期待した。

5. キャラクター設計

語り手は「ノア」という異世界から来た観測者である。ノアの故郷では、すべての存在が思念で直接つながっており、言葉も表情も嘘も存在しない。そのような世界から来たノアが、人間たちが「言葉」という不完全な道具で気持ちを伝えようとする姿を観察する、という設定にした。

この設定には意図がある。「人間ではない視点から人間を描く」ことで、読者にとって当たり前の日常が新鮮に見える効果を狙っている。また、観測者が徐々に人間に心を動かされていくという成長の軸を持たせることで、日記を読み続ける動機を作っている。

トーンとマナーとして以下を設定した。

- 詩的で少し物悲しい語り口調。
- 人間の常識、感情、社会のルールから微妙にズレた第三者の視点を持つ。
- 喜怒哀楽を直接的な表現で出さず、淡々と情景を描写する。
- 一人称は「私」。

6. 生成フロー

1日分の日記生成は以下の流れで行われる。

生成フローの全体像

```
main.py 実行
|
v
(1) 観測地点の決定 --- Google Maps API で住所を取得
|
v
(2) 過去記憶の読み込み
    - character.md (キャラ設定)
    - 過去の日記 (直近7日分)
    - knowledge.md (法則)
    - emotional.md (内面)
|
v
(3) プロンプト構築
|
v
(4) Claude API 呼び出し (claude-sonnet-4-5)
    | フェーズ判定 / 日記 / 法則 / 内面 を一括生成
    v
(5) 出力のパーズ & 保存
    -> dayX.md      保存
    -> knowledge.md 追記
    -> emotional.md 追記
    -> locations.json 更新
```

(1) 観測地点の決定

東京近郊のランダムな緯度経度を生成し、Google Maps Geocoding APIで住所に変換する。一定確率（10%）で過去に訪れた場所の近くを再訪する仕組みも導入した。

(2) コンテキストの構築

以下の情報をすべて読み込み、1つのプロンプトに組み立てる。

- キャラクター設定 (character.md)
- 直近7日分の過去の日記 (dayX.md)
- 蓄積された「人間の法則」 (knowledge.md)
- 観測者の内面記録 (emotional.md)
- 再訪時は該当日の過去の日記

(3) フェーズ判定

AIに対し、蓄積された内面記録を読み返させ、現在のキャラクターがどの成長フェーズにいるかを自己判定させる。

(4) 日記生成

Claude API (claude-sonnet-4-5) を呼び出し、4つのセクション（フェーズ判定、日記本文、人間の法則、内面記録）を一度に生成する。

(5) 出力のパーズと保存

生成結果をセクションごとに分割し、日記はdayX.mdとして保存、法則と内面記録はそれぞれのファイルに追記する。観測地点の座標もlocations.jsonに記録する。

7. 使用モデル・ツール・ライブラリ

- **LLM:** Claude Sonnet 4.5 (Anthropic)
 - temperature: 0.6 (創造性と一貫性のバランス)
 - max_tokens: 4096
- **API:** Google Maps Geocoding API (住所の逆引き)
- **言語:** Python 3
- **ライブラリ:** anthropic (Claude API SDK)、requests、python-dotenv
- **開発支援:** Claude Code (コード実装・設計の壁打ちに使用)

8. 工夫した点

(1) 外部記憶による連続性の実現

LLMはステートレスだが、生成のたびに過去の日記・法則・内面記録をプロンプトに注入することで、疑似的な「記憶」を実現した。これにより、過去の出来事を参照した描写や、以前立てた法則の修正といった連続的な物語が可能になった。

(2) フェーズ制による自律的なキャラクター成長

当初は「Day 1-2はこう、Day 3-4はこう」と日数ベースでキャラクターの感情段階を定義していたが、7日間で完結してしまうためスケラブルでなかった。そこで6段階のフェーズを定義し、各フェーズに「次への兆候」を記述する形に変更した。どのフェーズにいるかはAI自身が内面記録を読み返して判断する。これにより、日数に依存しない有機的な成長が可能になった。

(3) 再訪メカニズム

観測地点は毎回ランダムだが、10%の確率で過去に訪れた場所の近くを意図的に選ぶ仕組みを入れた。再訪時には過去の日記もプロンプトに含め、「あの日と同じ街の変化」を描写できるようにした。Haversine公式で2地点間の距離を計算し、2km以内なら「近い場所」と判定している。偶然近くになった場合も検知する。

(4) 法則の蓄積と修正

ノアは観察を通じて「人間の法則」を発見し、記録する。この法則は追加だけでなく、後日の観察で修正・深化される。例えば「法則002: 無目的な静止」はDay 1で発見され、Day 4で「対象を伴う静止」への修正が加わり、Day 6でさらに「内部への応答」として再修正された。この修正の連鎖が物語に厚みを与えている。

(5) 表現の制約による文学性

「感情を直接表現しない」「人間の感情を理解したと断言しない」といったルールを設けることで、かえって詩的で余韻のある文体が生まれた。制約が創造性を引き出す設計になっている。

9. うまくいった点（定量データを含む）

- **日記の連続性:** 過去の法則への言及や、前日との内面変化の比較が自然に行われ、1話完結ではなく続き物として読める日記になった。
- **キャラクターの一貫性:** トーン&マナーと絶対ルールの制約により、生成のたびにキャラクターがブレることがほとんどなかった。
- **内面変化の自然さ:** 7日間の日記では、Day 1の無機質な記録からDay 7の詩的な描写まで、グラデーション的に変化する様子が読み取れた。
- **法則の修正の面白さ:** 同じ法則が複数回修正されることで、ノアの認識の深まりが可視化され、読み物としての面白さが生まれた。

定量的な実績:

- **フェーズ遷移:** Day 1〜5はPhase 1（観測者）、Day 6でPhase 2（違和感）に遷移。7日間で1段階の進行であり、急すぎず停滞もしない自然なペースが実現できた。
- **法則の蓄積:** 7日間で11件の法則が蓄積された（1日あたり平均1.6件）。うち2件は後日修正された（法則001: 1回修正、法則002: 2回修正）。特に法則002「無目的な静止」はDay 1, 4, 6の3回にわたり認識が深化しており、最も修正回数が多い法則となった。
- **内面記録の変化:** Day 1では「空気が肺に入る感覚」のような物理的な記録だったが、Day 7では「ガラスに映った自分の輪郭」「観測者と観測対象の境界線」といった自己言及的な記述に変化しており、内省の深まりが確認できる。

10. 難しかった点

コンテキストの膨張: 日数が増えると法則ファイルや内面記録が大きくなり、プロンプトのトークン上限に近づく。現在は直近7日分の日記のみ渡す制限をかけているが、法則と内面記録は全件渡しているため、長期運用では要対策。

出力のパーズ: AIの出力を4セクションに分割する処理は、見出しの文字列一致に依存している。AIが見出しの書き方を微妙に変えた場合にパーズが失敗するリスクがある。

フェーズ進行の制御: AI自身にフェーズ判定を委ねているため、進行が速すぎたり停滞したりする可能性がある。「初めの7日間の間に1つはフェーズを変化させて」といった最低限のガイドを追加して対処した。

11. プロンプト設計の詳細

本システムの品質はプロンプト設計に大きく依存している。ここでは、実際に使用した設定ファイルとプロンプトの構造を具体的に示す。

(1) キャラクター設定ファイル（character.md）の構成

character.mdは以下の5つのセクションで構成されている。

- **属性:** ノアの出自と世界観の設定。「思念で直接つながる故郷」と「言葉という不完全な道具を使う人間世界」の対比を明示。
- **トーン&マナー:** 「詩的で少し物悲しい語り口調」「喜怒哀楽を直接的な表現で出さない」など、文体レベルの指示を4項目で定義。
- **目的と行動原則:** 毎日の行動ルール。観察・考察・非干渉の3点。

- **成長のフェーズ**: 6段階のフェーズ定義。各フェーズには「次への兆候」を記述し、AI自身が遷移タイミングを判断できるようにした。例えばPhase 1の兆候は「記録する『必要のない』ものを記録したくなった時」と定義されており、実際にDay 4の内面記録で「風の冷たさ、本を閉じる速度——これらは人間の生態の法則とは無関係だ。なのになぜ、記憶の中に残っているのか」という記述が生まれ、Day 6でPhase 2への遷移につながった。
- **絶対を守るルール**: 5つの制約ルール。「感情を『理解した』と断言してはならない」「行動を善悪で判断しない」など。これらの制約が、かえって間接的で文学的な表現を引き出す効果を持った。

(2) プロンプトの全体構造

1回の生成で渡すプロンプトは、以下の順序で構成される。

1. キャラクター設定（character.md全文）
2. 本日の情報（Day番号、観測地点の住所、再訪の手がかり）
3. これまでの全記録
 - 過去の観察日記（直近7日分）
 - 再訪地点の過去の日記（直近7日分に含まれない場合のみ）
 - 蓄積された人間の法則（knowledge.md全文）
 - 観測者の内面記録（emotional.md全文）
4. フェーズ判定指示（内面記録を読み返し、現在のフェーズを自己判定するよう指示）
5. 出力指示（4セクションの見出し・形式・注意点を厳密に指定）

出力は「フェーズ判定→日記本文→人間の法則→内面記録」の4セクションを1回のAPI呼び出しで同時に生成させる。セクションごとに別々に呼び出す方式も検討したが、フェーズ判定の結果が日記のトーンに直接影響するため、一括生成の方が整合性が高いと判断した。

(3) 内面記録（emotional.md）の設計意図

内面記録は単なる感情ログではなく、「核心的感覚」「気になった存在」「自問」「前日からの変化」の4項目を毎日記録させる構造にした。これにより、AIが次回のフェーズ判定で自分の変化を振り返る際に、具体的な手がかりを得られるようにしている。

12. リポジトリ構成と実行手順

(1) ディレクトリ構成

```
ai-observer-diary/
├── main.py           # メインスクリプト（生成フロー全体を制御）
├── cleanup.py        # データ初期化用スクリプト
├── requirements.txt  # 依存ライブラリ（anthropic, requests, python-dotenv）
├── .env              # APIキー（ANTHROPIC_API_KEY, GOOGLE_MAPS_API_KEY）
├── memory/
│   ├── character.md  # キャラクター設定（固定）
│   ├── knowledge.md  # 蓄積される人間の法則（追記型）
│   ├── emotional.md  # 観測者の内面記録（追記型）
│   ├── locations.json # 観測地点の座標履歴（JSON）
│   └── daily_log/
│       ├── day1.md   # 各日の日記
│       ├── day2.md
│       └── ...
```

(2) セットアップと実行手順

リポジトリをクローンする

Python仮想環境を作成し、依存ライブラリをインストールする

```
pip install -r requirements.txt
```

.envファイルにAPIキーを設定する

```
ANTHROPIC_API_KEY=sk-ant-...  
GOOGLE_MAPS_API_KEY=AIza...
```

main.pyを実行する（1回の実行で1日分の日記が生成される）

```
python main.py
```

翌日分を生成するには、再度main.pyを実行する。day番号は既存ファイルから自動で算出される。

(3) 初期状態に戻す場合

cleanup.pyを実行すると、daily_log/内の日記ファイルとlocations.jsonが削除され、knowledge.mdとemotional.mdが初期状態（ヘッダーのみ）にリセットされる。character.mdは変更されない。

13. 計算資源

- **Claude Sonnet 4.5 API**
- **Google Maps Geocoding API**: 1回の生成あたり1リクエスト。無料枠内で収まる。
- **ローカル環境**: Python実行のみで、GPUは不要。

14. 今後の改善余地

- **コンテキスト圧縮**: 法則や内面記録が一定量を超えた場合に、要約して渡す仕組みの導入。
- **読者向けUI**: 生成された日記をブログやWebサイトとして公開し、フェーズの進行を可視化する。